

Perceptrón Lineal — Explicación del Código

Ejercicio 0 (Validación) — TP3 Sistemas de Inteligencia Artificial

2026

Introducción

Este documento explica el funcionamiento del código implementado para el **Ejercicio 0 (Validación)** del TP3. El objetivo es validar la implementación de un perceptrón lineal (Adaline) ajustando 50 puntos generados a partir de la función $y = 3x + 2$ con ruido gaussiano.

El proyecto consta de dos scripts independientes:

1. `generate_linear_dataset.py` — genera el dataset sintético.
2. `linear_perceptron.py` — entrena el perceptrón y produce resultados.

Generación del Dataset

`generate_linear_dataset.py`

La función `generate_dataset(n_points, seed)` genera puntos que siguen una relación lineal con ruido:

$$y = 3x + 2 + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 0.5)$$

Los valores de x se muestrean uniformemente en el intervalo $[-10, 10]$.

Parámetros CLI

Flag	Default	Descripción
<code>--output</code>	<code>linear_dataset.csv</code>	Ruta del CSV de salida
<code>--n_points</code>	50	Cantidad de puntos a generar
<code>--seed</code>	None	Semilla para reproducibilidad

Ejemplo de uso

```
python generate_linear_dataset.py --output linear_dataset.csv --seed 42
```

Esto produce un CSV con dos columnas (x , y) y 50 filas.

Perceptrón Lineal (Adaline)

Fundamento teórico

El perceptrón lineal utiliza la **función de activación identidad**: la salida O es directamente el estado de excitación h .

$$O^\mu = h^\mu = \sum_{i=0}^n w_i x_i^\mu$$

Donde $x_0 = 1$ para todos los datos (truco del bias), de modo que w_0 actúa como el sesgo (bias) y w_1 como la pendiente.

Regla de actualización (online learning)

Para cada dato μ , los pesos se actualizan inmediatamente:

$$\begin{aligned}\Delta w_i &= \eta (z^\mu - O^\mu) x_i^\mu \\ w_i &\leftarrow w_i + \Delta w_i\end{aligned}$$

Donde η es la tasa de aprendizaje y z^μ es la salida esperada.

Función de error

Al final de cada época, se calcula el **Error Cuadrático Medio (MSE)** sobre todo el dataset:

$$\text{MSE} = \frac{1}{P} \sum_{\mu=1}^P (z^\mu - O^\mu)^2$$

Criterio de corte

El entrenamiento se detiene cuando:

- El MSE cae por debajo de un umbral ε , o
- Se alcanza la cantidad máxima de épocas.

linear_perceptron.py — Estructura del código

Función `train_perceptron(df, learning_rate, epochs, epsilon)`

Es el núcleo del algoritmo. Paso a paso:

1. **Extracción de datos**: separa x (feature) y y (salida esperada z^μ) del DataFrame.
2. **Bias trick**: agrega una columna de unos ($x_0 = 1$) al inicio de la matriz de entrada. La matriz X queda con forma $(P, 2)$.
3. **Inicialización de pesos**: se generan w_0 y w_1 como valores aleatorios pequeños en $[-0.1, 0.1]$.
4. **Bucle de entrenamiento (por época)**:
 - Para cada dato μ : calcula la salida $O^\mu = \mathbf{w} \cdot \mathbf{x}^\mu$, calcula el error, actualiza los pesos.
 - Calcula el MSE sobre todo el dataset.

- Si $\text{MSE} < \varepsilon$: corta el entrenamiento.
5. **Retorna** el vector de pesos y el historial de MSE por época.

Función `run_and_save(csv_path, learning_rate, epochs, epsilon, output_dir)`

Orquesta el pipeline completo:

1. Carga el CSV con pandas.
2. Llama a `train_perceptron`.
3. Crea la carpeta de output.
4. Guarda `weights.csv` con los pesos finales (w_0, w_1) y el MSE final.
5. Genera `plot.png`: un scatter plot de los datos con la recta aprendida superpuesta.

CLI (main)

Flag	Default	Descripción
<code>--csv</code>	(requerido)	Path al CSV de entrada
<code>--learning_rate</code>	0.01	Tasa de aprendizaje η
<code>--epochs</code>	1000	Cantidad máxima de épocas
<code>--epsilon</code>	10^{-4}	Umbral de convergencia (MSE)

La carpeta de output se nombra automáticamente como `output_{nombre_csv}_{timestamp}`.

Ejemplo de uso

```
python linear_perceptron.py --csv linear_dataset.csv \
    --learning_rate 0.001 --epochs 2000 --epsilon 1e-4
```

Resultados de la validación

Con los parámetros `learning_rate=0.001`, `epochs=2000`, `epsilon=1e-4` y seed 42:

Parámetro	Valor esperado	Valor obtenido
w_0 (bias / ordenada al origen)	≈ 2	1.905
w_1 (pendiente)	≈ 3	3.007
MSE final	bajo	0.144

El MSE no llega a 0 porque el dataset incluye ruido gaussiano ($\sigma = 0.5$). Un MSE de ~ 0.14 es consistente con la varianza del ruido ($\sigma^2 = 0.25$).

Tests

Se incluyen 7 tests automatizados en `tests/`:

Dataset (`test_generate_dataset.py`):

- Forma y columnas correctas (50 filas, columnas `x` e `y`).

- Rango de x dentro de $[-10, 10]$.
- Tendencia lineal: coeficientes de un ajuste con `polyfit` cercanos a 3 y 2.

Perceptrón (`test_linear_perceptron.py`):

- Aprende los pesos correctos con datos exactos (sin ruido).
- El MSE decrece a lo largo del entrenamiento.
- Convergencia temprana (early stop) cuando los datos lo permiten.
- Pipeline completo genera `weights.csv` y `plot.png`.